

CSCE 638 NATURAL LANGUAGE PROCESSING · MODULE 05 · LECTURE NOTES

Sequence Models

Recurrence, and the problem it could not solve.

MODULE OUTCOMES

- Explain the recurrent formulation and its appeal.
- Explain vanishing gradients and why depth in time is different.
- Explain gating and what it buys.
- Explain the remaining limitations that attention fixed.
- Demonstrate a long-range failure experimentally.

1 Recurrence

$$\mathbf{h}_t = \sigma(\mathbf{w}_h \mathbf{h}_{t-1} + \mathbf{w}_x \mathbf{x}_t + \mathbf{b})$$

- **One hidden state, updated by the same weights at every step** — and **the state is the entire memory** of everything seen so far, which is §1's callout.
- **The output is computed from the current state**, so a prediction is available at every position, which makes the formulation natural for tagging and for language modelling alike.
- **And the parameters are independent of the sequence length** — **which is the appeal: one set of weights handles any length, and shares statistical strength across all positions** rather than learning each position separately.
- **Weight sharing across time is the point**, and it is worth connecting to CSCE 636 Module 05's **convolution, which shares weights across space** — **the same idea on a different axis**, with the same justification: the pattern you are looking for can occur anywhere, so the detector should not depend on position.

THE STATE IS A FIXED-SIZE SUMMARY OF AN UNBOUNDED PAST, WHICH IS THE WHOLE PROBLEM

Everything the model knows about the first three hundred tokens must fit into one vector — and that vector is overwritten, at least partially, at every single step.

Which is a hard information bottleneck rather than a tuning issue. A fixed-dimensional state cannot retain arbitrary amounts of detail about an arbitrarily long prefix, by counting, so something must be discarded at every step.

And the model has to learn *what* to discard — which it does imperfectly, and which **depends on information about the future that it does not have**: whether a detail matters depends on what is asked later.

So recurrence is theoretically sufficient and practically limited. A recurrent network is a universal sequence approximator and still cannot be *trained* to use its own capacity over long distances — and that gap between representational capacity and trainability is §2's subject and one of the most instructive patterns in the whole of deep learning (CSCE 636 Module 04).

2 Vanishing gradients

Backpropagating from step t to step $t-k$ multiplies by the recurrent Jacobian k times:

$$dL/dh_{\{t-k\}} \sim (W_h)^k \times (\text{activation derivatives})^k$$

SO

largest eigenvalue < 1 $\rightarrow$ the gradient vanishes
largest eigenvalue > 1 $\rightarrow$ the gradient explodes

AND NEITHER IS A STABLE MIDDLE. The exponent is the distance in time, so a 100-step dependency needs 100 multiplications for the signal to survive.

THE PARTIAL FIXES

gradient clipping stops exploding
careful initialisation delays vanishing
shorter truncation windows cheaper, and gives up
the long range entirely
and gating (section 3), which is the real one

COMPARE CSCE 636 Module 04: a residual connection fixes depth in LAYERS by adding an identity path. Gating is the same move, along the time axis.

The exponent is the distance in time, which is why this is qualitatively worse than ordinary depth: a fifty-layer network has a fixed and manageable depth, whereas a sequence model on a thousand-token input has an effective depth of a thousand, with the same multiplicative structure. **The problem scales with the input rather than with the architecture**, which is what makes it the binding constraint.

3 Gating

AN LSTM ADDS A CELL STATE THE NETWORK CAN CARRY FORWARD UNCHANGED

The forget gate decides what to drop from the cell state, the input gate decides what to add, and the output gate decides what to expose to the rest of the network — all three learned, and all conditioned on the current input and the previous state.

And crucially, the cell state's update is *additive* — which gives the gradient a path with a multiplier near one, so it can travel a long way without vanishing, because adding does not shrink the signal the way repeated multiplication does.

Which is exactly the residual connection argument (CSCE 636 Module 04) applied along the time axis instead of the layer axis — and recognising that is what makes gating memorable rather than an arbitrary arrangement of sigmoids. The LSTM predates ResNets by nearly two decades and contains the same insight.

GRUs achieve much the same with fewer gates and perform comparably across most tasks — which strongly suggests that the additive path was what mattered and that the specific gate arrangement mattered considerably less, a conclusion the ablation literature supports.

- Sequential computation. Step t requires step $t-1$, so the sequence cannot be parallelised during training — which directly caps the model size and data volume you can afford, and was the binding constraint on scale before 2017.
- The bottleneck is relieved rather than removed. Long dependencies still degrade, merely more slowly — and the degradation is gradual rather than cliff-edged, which makes it easy to miss in aggregate metrics and is why §4's constructed task matters.
- And the path length between two positions remains $O(\text{distance})$ — so the model must route information through every single intervening step, each of which can corrupt or discard it.
- Which is the specific thing attention fixes (Module 06): a constant-length path between any two positions, regardless of how far apart they are.
- And parallelism, as a second and entirely unrelated benefit. The two benefits of attention are independent: constant path length addresses quality, and parallelism addresses scale — and the second is arguably why it won, since it unlocked the data and parameter scales that Module 07 depends on.

4 Demonstrating the failure

THE COPY TASK

input: a random sequence, a delimiter, then blanks
 target: reproduce the sequence after the delimiter
 vary the gap. Accuracy against gap length is the measurement.

WHY IT ISOLATES THE PROPERTY

nothing about the task is hard except carrying information across the gap. No syntax, no semantics, no ambiguity, no world knowledge.

WHAT YOU SHOULD SEE

a plain RNN fails at quite modest gaps
 an LSTM succeeds much further and then degrades
 a single attention layer handles it flatly

AND THIS IS PROJECT 1'S REQUIREMENT -- construct the task, measure the curve, and report where each model breaks. Asserting the failure is not enough; the construction is what is graded.

AND WHERE RECURRENCE STILL APPEARS

Streaming and online settings, where you cannot see the whole sequence before producing output — **recurrence is naturally incremental and attention is not**, which matters for real-time transcription and for very low-latency applications.

Very long sequences, where attention's quadratic cost in length becomes prohibitive — **which is precisely why the state-space and linear-attention model families revived recurrent ideas**, and did so with the gradient problem addressed by construction rather than by gating.

Small models on constrained hardware, where the parameter count and the memory cost matter more than the last increment of quality — a recurrent model's memory does not grow with the sequence length, and attention's does.

So the honest position is that attention replaced recurrence for the dominant case rather than for all cases — and **the recurrent formulation is worth understanding because the arguments recur**, literally: the bottleneck, the gradient path, and the parallelism trade all reappear whenever somebody proposes an alternative to attention.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Jurafsky & Martin, chapter 9 (free)	§1 through §3, with the LSTM equations written out carefully.

Resource	What it covers for this module
Olah — Understanding LSTM Networks (free)	§3 as the clearest available explanation — the diagrams are what make the gates click.
Pascanu et al. — On the difficulty of training RNNs (free)	§2's analysis in the original , including the clipping argument and the eigenvalue condition.
Hochreiter & Schmidhuber — Long Short-Term Memory (free)	§3 in the original , from 1997 — and the constant-error-carousel framing is the additive path argument stated plainly.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement a plain RNN** and train it on a short sequence task.
- 2 **Plot the gradient norm against time distance** and observe it.
- 3 **Add gradient clipping** and report which problem it fixes and which it does not.
- 4 **Implement an LSTM** and identify the additive path in your own code.
- 5 **Explain the parallel to residual connections** in your own words.
- 6 **Build the copy task** with a configurable gap.
- 7 **Measure accuracy against gap length** for RNN, LSTM, and attention.
- 8 **Report where each breaks**, with the curve.
- 9 **Time one training epoch** for a recurrent and an attention model on the same data.
- 10 **Explain why the recurrent one cannot be parallelised.**

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the recurrent formulation and say what its appeal is.
- 2 What does recurrence share with convolution?
- 3 Why is a fixed-size state a hard bottleneck?
- 4 What is the gap between capacity and trainability?
- 5 Why does the gradient vanish or explode, and what is the exponent?
- 6 Why is depth in time worse than depth in layers?
- 7 What do the three LSTM gates do, and which property matters most?

- 8 What does gating share with residual connections?
- 9 Name three things gating did not fix.
- 10 Why does the copy task isolate the property, and where does recurrence still appear?